

Chapter 1

Embedded Software Design B — Architectural Design

ENEL712 Embedded Systems Design — Lecture 9

Summary

Software architectures for embedded systems: bottom-up vs top-down development, polling vs event-driven vs state machines vs RTOS, hardware/software partitioning, fault tolerance, and documentation.

Topics Covered

- Embedded software development fundamentals
- Development approaches (bottom-up vs top-down)
- Software architectures (polling, event-driven, FSM, OS, RTOS)
- Hardware/software partitioning
- Fault tolerance and defensive design
- Software documentation

1.1 Embedded Software Development Fundamentals

- Iterative process — designs are rarely correct first time, regardless of experience.
- Skills accrue through projects: C/C++ proficiency, peripheral knowledge, reusable templates.
- Tool selection depends on project requirements and engineer experience — no universal solution.
- Dominant languages: C (~82 %), C++ (~45 %), Assembly (~35 %).

1.2 Development Approaches

Approach	Pros	Cons
Bottom-Up	Quick coding; immediate hardware testing	No clear system picture; undefined interfaces; complexity creep
Top-Down	Architecture is defined first; fewer late changes; low-level modules become clear	Testing only starts mid-project

Starting any project without a clear architectural design is poor form and risky.

1.3 Software Architectures

1.3.1 Polling Loop

- Simplest strategy — loop checks all inputs and handles all actions.
- Delays stall the processor; wasted cycles, poor power efficiency.

1.3.2 Event-Driven

- Internal data events or external events (interrupts) trigger processing.
- CPU is free between events — can do other tasks or sleep.
- Hybrid systems often use interrupts for high-priority events plus polling for low-priority ones.

1.3.3 Finite State Machine (FSM)

- Program divided into a finite set of states; transitions on internal/external events.
- Widely used; typically implemented with a switch-case statement.

1.3.4 Operating System (OS)

- Manages resources and provides services for applications; embedded OSs are often cut-down PC OSs.
- Kernel is the core of the OS controlling networking, file systems, etc.
- Provides apparent parallelism via time-slicing; standard OSs do not guarantee timing.

1.3.5 Real-Time Operating System (RTOS)

- Designed for time-critical apps with deterministic execution.
- Soft real-time: timing not critical; Hard real-time: must strictly meet timing.
- Examples: FreeRTOS, SYS/BIOS. Standard Windows/Linux are not RTOSs.

1.4 Hardware/Software Partitioning

- Hardware: faster development for performance tasks; runs concurrently with the CPU; use built-in peripherals (DMA, Event System, comms ports).
- Software: lower per-device manufacturing cost; reproduction cost is just the memory used.

- Examples: switch debouncing, comms protocols (HW peripheral or SW bit-banging), encoding/decoding, encryption/decryption.

1.5 Fault Tolerance and Defensive Design

1.5.1 Fault tolerance techniques

- Power-On Self-Test (POST): verify components and sensible memory at startup.
- Checksums: detect corruption in stored data (e.g., EEPROM settings).
- Hardware initialisation: explicitly initialise all ports — never rely on datasheet defaults.

1.5.2 Defensive software

- Watchdog Timer to recover from runaway code.
- Boundary checks on variables.
- Fill unused memory with a safe routine (NOP).
- Define unused interrupt vectors to point to a safe routine.
- Disable interrupts while modifying hardware registers.

1.6 Software Documentation

- Critical for maintainability and often a legal requirement; do it as the project progresses.
- Sensible naming conventions; thorough comments.
- Graphical aids: state diagrams, flow charts, UML.
- Mathematical equations and technical manuals.

1.7 Use Cases & Applications

Architecture	Choose when...	Example
Polling loop	Tiny system; few inputs; latency » loop period	Bench LED + button demo
Event-driven (interrupts)	Latency-critical events; CPU should sleep	Battery sensor node, button debouncer
FSM (switch-case)	Discrete process with clear states & transitions	USB enumeration, vending machine controller
RTOS	Multiple concurrent activities with soft/hard deadlines	Drone flight stack, multi-sensor logger
Full OS (Linux)	Networking, storage, GUI, lots of throughput	Smart home gateway, automotive infotainment

1.8 Worked Examples

1.8.1 Polling vs interrupts for a button

A debounced button press takes ~10 ms. A polling loop running every 100 ms can miss a quick press. An interrupt-driven design starts a debounce timer on the GPIO change and reliably

catches every press while letting the CPU sleep between events.

1.8.2 Watchdog timer setup

Set the watchdog timeout $> 2 \times$ longest expected loop. If the main loop usually completes in 50 ms, a 250 ms watchdog gives margin for slow paths but still resets within a quarter-second of a runaway.

1.9 Code Snippets

1.9.1 Switch-case FSM

```
typedef enum { IDLE, ARMED, FIRING, COOLDOWN } state_t;
static state_t state = IDLE;

void step(event_t e) {
    switch (state) {
        case IDLE:      if (e == ARM)      state = ARMED;      break;
        case ARMED:     if (e == TRIGGER) { fire();  state = FIRING;  }
                        else if (e == DISARM) state = IDLE;      break;
        case FIRING:    if (e == DONE)      state = COOLDOWN; break;
        case COOLDOWN:  if (e == COOL)      state = IDLE;      break;
    }
}
```

1.9.2 POST + checksum on settings

```
// Verify that EEPROM-stored settings are valid before using them.
uint8_t computed = crc8(settings, sizeof(settings) - 1);
if (computed != settings.crc) {
    load_factory_defaults();
    save_settings();
}
```

Key Definitions

Bottom-Up Development

Start with low-level modules; quick to test but lacks system picture.

Top-Down Development

Start with high-level architecture; fewer late surprises but slower to test.

Polling Loop

Repeatedly check inputs and run actions; wastes cycles when delays are needed.

Event-Driven Architecture

Triggered by interrupts/events; CPU can sleep or do other work in between.

FSM

Finite State Machine; states and transitions, often a switch-case implementation.

RTOS

Real-Time Operating System; deterministic, soft or hard real-time.

Fault Tolerance

Ability to recover automatically from errors (e.g., power spikes).

Defensive Design

Techniques limiting damage from interference (boundary checks, WDT, safe vectors).

Watchdog Timer

Resets the MCU if the program fails to pet it periodically.

POST

Power-On Self-Test; checks components and memory sanity at startup.

Soft Real-Time

Timing requirements where missing a deadline degrades quality (e.g., audio glitches) but isn't catastrophic.

Hard Real-Time

Timing requirements where a missed deadline is a system failure (e.g., engine ignition timing).

Brown-Out Detection

MCU feature that holds the CPU in reset when supply voltage dips below a threshold, preventing erratic behaviour.

Re-entrant Function

Function that can be safely interrupted and called again from the ISR without corrupting state (no static variables, no shared globals).

Sources

- Lecture 9.pdf